# CS111 Fall 2024

Discussion (week 2)

Jui-Nan Yen

# Agenda

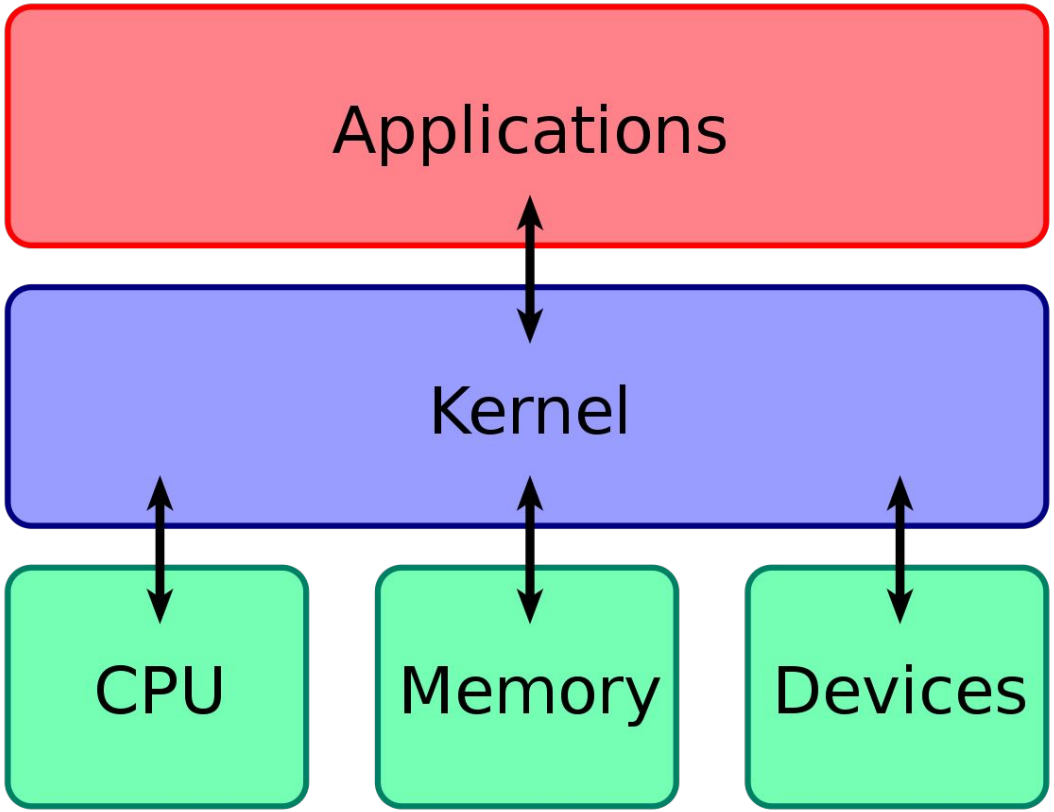
- Lab0
- Git refresher
- Q&A

# Lab0

# Kernel

Lowest level of the OS.

- We have multiple modes of execution in computer systems, each defines some level of rights and privileges (i.e, what resource can you access).
- Library functions run in user mode, which has the least privileges. However, the system calls run in kernel mode, as those functions are part of the kernel itself.




**Q: Why is this extra layer of indirection necessary?**

# What’s kernel module

## Kernel modules [2]:

- Are pieces of code that can be loaded and unloaded into the kernel upon demand.
- They extend the functionality of the kernel without the need to reboot the system.
- Why not just add all the new functionalities into the kernel image on demand?

  It’s bad. Large kernel size, rebuild and reboot kernel upon every change

Modules are stored in `/usr/lib/modules/kernel_release`

To see what kernel modules are currently loaded:

```
lsmod
```

To show information about a module:

```
modinfo <module_name>

//Example:

modinfo proc_count
```

# Kernel Modules

We use many libraries when writing user level applications.

- For example, printf() is a handy function provided by standard C library, the actual definition of this function will enter your code during the linking stage.

However, at kernel level, we don’t have access to those functions, because the modules are object files whose symbols get resolved upon insmod step.

The definition for the symbols comes from the kernel itself; there are some external libraries provided by kernel, which you can use. Take a look at: /proc/kallsyms

# Kernel Modules

To load a module:

```
sudo insmod <module_name> [args]

//example:

sudo insmod proc_count.ko
```

To unload a module:

```
sudo rmmod <module_name>

//example:

sudo rmmod proc_count
```

# Kernel Modules

Kernel modules must have at least two functions:

- A “start” initialization function called `init_module()`, which is called when the module is insmoded into kernel
- A “end” cleanup function called `cleanup_module()` , which is called just before it is rmmoded.

Every kernel module needs to include “linux/module.h” header file

***Note: in newer versions, you can use whatever name you like for the start and the end functions, you just need register them using `module_init()` and `module_exit()` macros.***

# Kernel Modules

How do we print things out?

printk():

- was not meant to communicate information to the user
- A logging mechanism for kernel(used to log information or give warnings)
- Each printk() comes with a priority

| | | |
|---|---|---|
| 0 | KERN_EMERG | An emergency condition; the system is probably dead |
| 1 | KERN_ALERT | A problem that requires immediate attention |
| 2 | KERN_CRIT | A critical condition |
| 3 | KERN_ERR | An error |
| 4 | KERN_WARNING | A warning |
| 5 | KERN_NOTICE | A normal, but perhaps noteworthy, condition |
| 6 | KERN_INFO | An informational message |
| 7 | KERN_DEBUG | A debug message, typically superfluous |

Example:

```
printk(KERN_INFO "Hello Kernel \n");
```

***Note: you need to include <linux/kernel.h> to use the above macros***

# Kernel Modules: example 1 [3]

```
#include <linux/module.h>     /* Needed by all modules */

#include <linux/kernel.h>     /* Needed for KERN_INFO */

int init_module(void)

{

      printk(KERN_INFO "Hello world 1.\n");

      return 0;

}

void cleanup_module(void)

{

      printk(KERN_INFO "Goodbye world 1.\n");

}
```

# Kernel Modules

**But is there any better way to print things out ?**

# Sequence Files

- Writing a /proc file may be quite "complex".
- To help people write /proc file, there is an API named **seq_file** that helps format a /proc file for output. It's based on sequence, which is composed of 3 functions: start(), next(), stop(), and show(). The **seq_file** API starts a sequence when a user read the /proc file. This comes handy, when you need to output something really big.
- This API is mainly for reading very large data in a convenient way (you can maintain the sequence state information easily). However, we only use one of its utility functions to make our life easier for printing, as we can easily format and output data to the user without worrying about things like output buffers. [5]
- For our purposes, we are printing out a simple(i.e, a small) output, so we only need the show() function.

*If you're super curious about the internals of sequence files, check this out.*

# Show function example

**seq_printf**: works pretty much like printk(), but requires the seq_file pointer as an argument

```
static int seq_show(struct seq_file *s, void *v)

{

        int my_num_to_show = 1;

        seq_printf(s, "%d\n", my_num_to_show);

        return 0;

}
```

*Note the signature of the show function, if your arguments types are incorrect, you'll have errors.*

***Note: This function needs to return zero, if everything is correct. Otherwise, you'll have issues.***

# /proc directory: control and information center for kernel

Originally designed to allow easy access to information about processes [4]

Try a few commands to see what they do:

```
cat /proc/modules

cat /proc/meminfo
```

# /proc directory: control and information center for kernel

Virtual- file system: doesn't contain real files on the disk

- Rather: contains runtime system information that is generated on the spot, and is located in memory.
- In fact, you can get details about all running processes in /proc, it is the closest thing to the process table that you can access
- A lot of system utilities are calls to files in this directory
  - For example:
    - `lsmod → cat /proc/modules` (note: /proc/modules is only populated when you *cat* the file)
  - Most files in /proc have a zero byte size, they are just pointers to the location of information
- insmod would make /proc/count appear
  - Code will only run when you do cat /proc/count
- rmmode should make /proc/count disappear

# How do we actually create /proc/count

The actual (i.e, the functional) /proc/count is created when the module is loaded with the function: `proc_create_single()`

This function returns a : `struct proc_dir_entry *`

Which is used to configure the file /proc/count

# proc_create_single()

```
proc_create_single(name, mode, parent, show)
```

name: name of the process file we are creating, for example: “count”

mode: permission mode

parent: parent of this process

show: show function you want to use

# proc_create_single()

```
//Example:

static struct proc_dir_entry * my_new_entry;

my_new_entry = proc_create_single(“count”,0644, NULL,my_show_function);
```

*Think about what each argument means here*

# How to count the number of processes?

`for_each_process(t)` iterates through all the active processes, it basically is  a simple for loop under the hood.

It takes `struct task_struct*t as an argument`

## What is a task struct ?

It represents a process descriptor , which contains all the information that the kernel has and needs about a process [6].  A task struct is a node in the doubly linked list, called task list, which is a list of information about the active processes.

# Make sure to clean up before exit

You can use proc_remove to clean up that new directory entry created by proc_create_single

# Kernel Modules : overview

**After writing your code, you need to compile it and load it into the kernel.**

To compile your code:

- cd into your project directory
- type: make

To load your compiled code into the kernel modules ( in your project directory):

- `sudo insmod proc_count.ko`

To check if the module was loaded correctly:

- `lsmod` `// this should list out all the loaded modules, so you should be able to see proc_count in that list`

If you want to change your code and recompile, before loading the newly compiled module, you will need to remove the old module from the system:

- `sudo rmmod proc_count`

Then you can follow the previous steps to load it back in.

*All the modules loaded into the kernel are listed in `/proc/modules` file. You can inspect this file to see if your module is loaded or not*

# Your Skelton Code

```
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/sched.h>

//your show function

static int __init proc_count_init(void)
{
        //your show function registration

        pr_info("proc_count: init\n");
        return 0;
}

static void __exit proc_count_exit(void)
{
        //clean up work here in this function
        pr_info("proc_count: exit\n");
}

module_init(proc_count_init);
module_exit(proc_count_exit);

MODULE_AUTHOR("Your name");
MODULE_DESCRIPTION("One sentence description");
MODULE_LICENSE("GPL");
```

# How to approach?

- First try to write and compile a module, which simply outputs a dummy string
- Then count the number of processes to output the actual process count

# Acknowledgement

[1] https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

[2] https://wiki.archlinux.org/title/Kernel_module

[3] https://tldp.org/LDP/lkmpg/2.6/html/lkmpg.html#HELLO2

[4] https://linux.die.net/lkmpg/x710.html

[5]https://www.kernel.org/doc/html/latest/filesystems/seq_file.html

[6]http://books.gigatux.nl/mirror/kerneldevelopment/0672327201/ch03lev1sec1.html

# Git Review

# Version Control

## What is version control ?

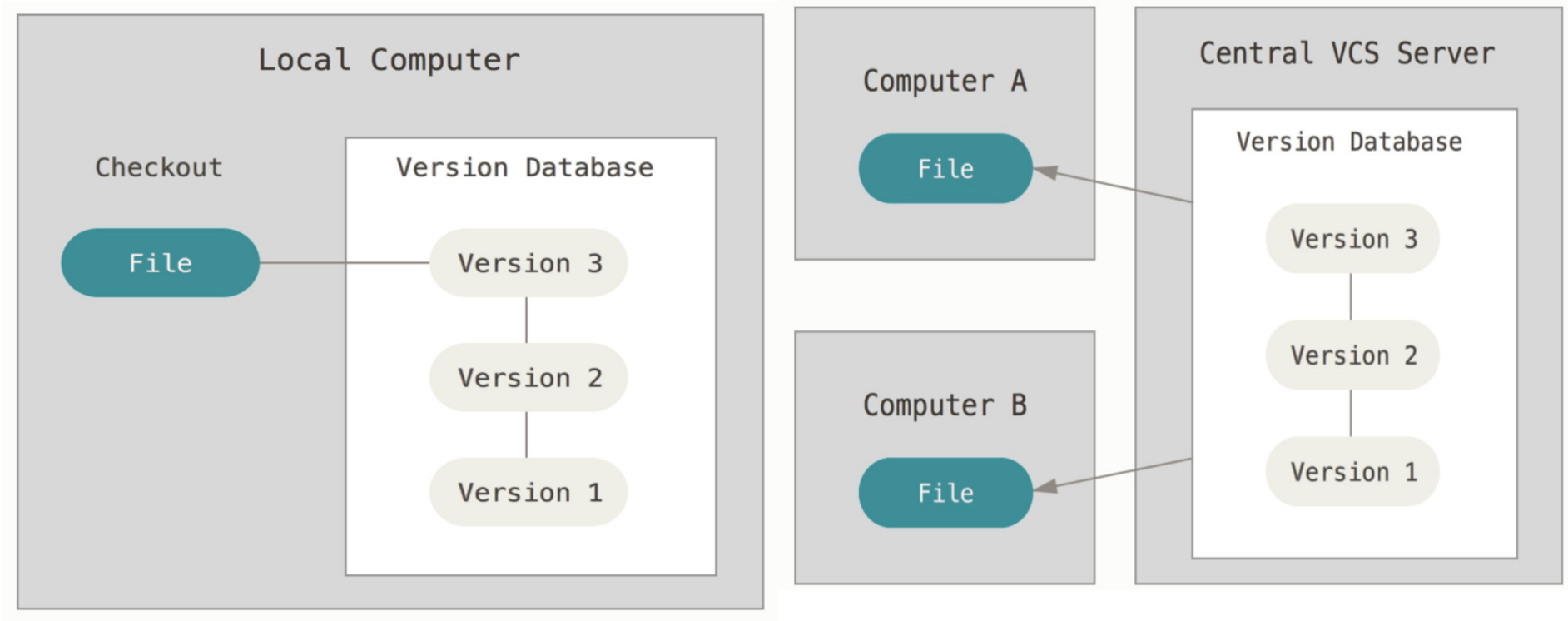
Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later [1].

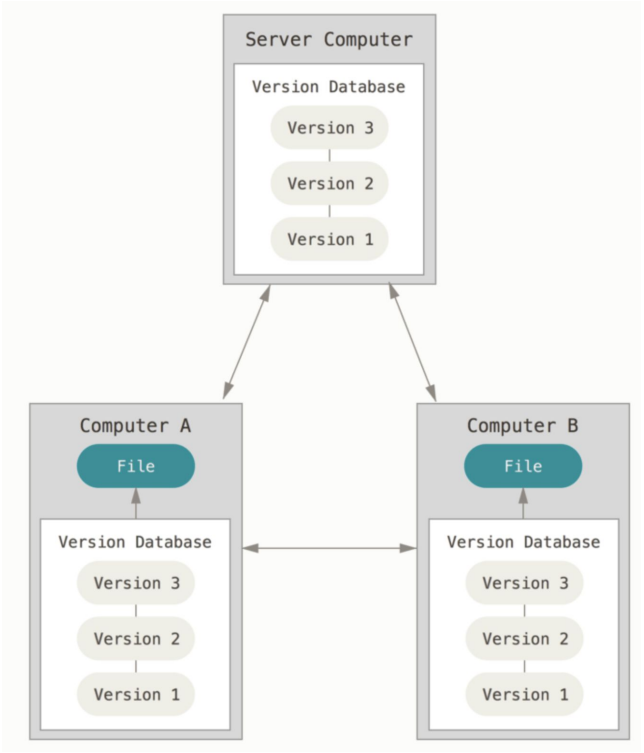
- It allows you to revert files/projects back to previous states
- Make comparisons between different versions
- Trace some issues, such as the last modification which caused an error, and when, by whom this modification was introduced, etc.
- Recovering back to a correct state, if you screw things up

**Git** is a version control tool, there are other VC tools as well such as,

# Approaches to version control

1. Local “naive” version control
    a. Keep track of multiple versions of the same directory (maybe timestamped)
        i. What is the issue here?
2. Local “smarter” version control
    a. Have a database of different versions (centralized)
        i. What issues do you see here?
3. Centralized version control
    a. Have a database of different versions(centralized), where multiple computers on the network can access the latest snapshot
        i. What issues?
4. Distributed version control
    a. Each server on the network  keeps a full copy of the files/project
    b. This is where Git sits


Local Computer
Checkout
File
Version Database
Version 3
Version 2
Version 1
Computer A
File
Computer B
File
Central VCS Server
Version Database
Version 3
Version 2
Version 1


Server Computer
Version Database
Version 3
Version 2
Version 1
Computer A
File
Version Database
Version 3
Version 2
Version 1
Computer B
File
Version Database
Version 3
Version 2
Version 1

# Git states

- Modified
  - You have changed the file but have not committed it to your database yet
- Staged
  - You have marked a modified file in its current version to go into your next commit snapshot
- Committed
  - The data is safely stored in your **local** database ( you'll still need to push it to a remote server, if that's needed )

# Git Workflow

1. You modify files in your working tree.
2. You selectively stage just those changes you want to be part of your next commit, which adds ***only*** those changes to the staging area.
3. You do a commit, which takes the files as they are in the staging area and stores that snapshot permanently to your Git directory.

# Getting started

There are GUI tools to use Git, but we do not recommend or support them.

Instead we stick with the command line interface, as it gives us all the commands available on Git, while the GUI tools only support a subset of them.

- Check if you have Git installed:
  `git --version`
- If not, download and install it from the Git download site
  - You can also download the binary manually and compile+install it manually ( which requires some dependency libraries, such as *autotools, curl, zlip, openssl, expat and libincov, etc*)

# Getting started Cont

Git has a powerful tool called config, which lets you get and set configuration variables that control how Git operates. These variables can be stored in three places:

1. `/etc/gitconfig` : values stored in this file applies to all the users on the systems and their repos. This requires superuser privilege to update.
    a. `git config --system ...`
2. `~/.gitconfig or ~/.config/git/config file:` values stored in this file applies to the current user, this affects all of the repositories the current user work with on the system.
    a. `git config --global ...`
3. `config in the Git directory:` specific to that single repository, and by default this config file is used, or you can make more explicit:
    a. `git config --local ...`

**The more specific level dominates !**

# Git Configuration

You need to set your username and email address after installing git, every git commit you make uses this information:

```
git config --global user.name “Salekh”

git config --global user.email salekh@cs.ucla.edu
```

You can also set up your favorite text editor, for more click here:

```
git config --global core.editor "code --wait"
```

Note: here the *global* option makes sure that you will only need to run this configuration command once, if you want different names and emails for each project you may want to omit the *global* option

# Git Configuration

You can check the settings as follows:

```
git config --list

git config --list --show-origin

//To check a specific key:

git config user.name

git config --show-origin user.name
```

# Getting a repository

Typically a repository is obtained in two ways:

1. Take a local directory which is not under version control, and turn it into a Git repository, or
2. Clone an existing Git repository from somewhere else

# Method 1: turning a directory into a Git repository

```
cd /Users/user/my_project_dir

git init
```

This will create a new subdirectory in the current directory called: *.git*

This directory contains all the necessary repository files, i.e, a Git repository skelton

# Method 2: cloning an existing repository

```
git clone <repo_url>

//Example:

git clone https://github.com/libgit2/libgit2

git clone git@laforge.cs.ucla.edu:summer21/username/cs111

git clone git@laforge.cs.ucla.edu:summer21/username/cs111 my_cs111
```

For example, when you clone the cs111 project, this command creates a directory named cs111, initializes a *.git* directory inside it, pulls down all the data for that repository, and checks out a working copy of the latest version. If you use the last command, it also renames the target directory as specified.

# Working with a repository

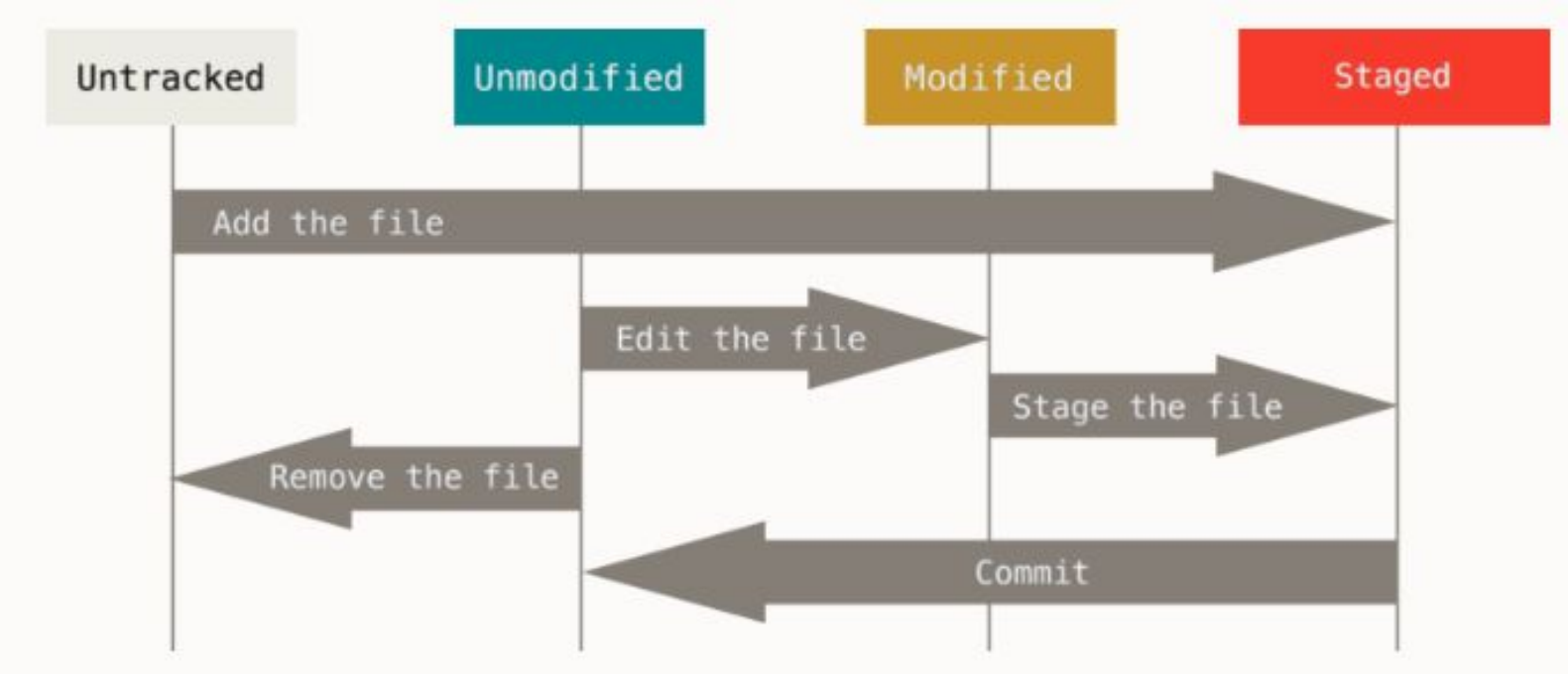
Once you have a repository set up, you can start working on it(i.e, making changes)

Each file in the working directory can be in either of the following states:

- Tracked
  - File is known to the Git, i.e, it was in the last snapshot
  - It can be modified, unmodified, or staged
- Untracked
  - Everything else, i.e, all the files in the working directory not included in the last snapshot and are not in the staging area

***When you first clone a repo, all the files will be tracked and unmodified. As you edit the files, you selectively stage these modified files and then commit all those staged changed.***

# Lifecycle of the status of the files



# File status

```
git status

git status --short/-s  //more compact status output

git add file_name
```

git add: add this content to the next commit, or start tracking this file (if the file was untracked previously)

Flags on short status: M: modified and staged, A: staged, MM: modified staged modified, ??: untracked

# Tracking the changes

*git status* is good, however, it only tells what files were changed. Sometime we may wanna know how they were changed exactly.

```
git diff

git diff --staged/--cached
```

git diff:  shows the changes in the file that are staged and the changes that are unstaged
git diff --staged/cached: compares the staged changes to your last commit
*You can also use git difftool to make this experience better with other tools*

# Commit

```
git commit

git commit -m “my commit message”

git commit -a -m “my commit message”
```

Only the files which are staged will be committed, anything that is unstaged, any files you have created or modified that you haven’t run git add on since you edited them won’t go into this commit.

git commit: will open a text editor window to let you type your commit message

git commit -m “message”: inline shorthand, skips the text editor window

git commit -a -m “my commit message”: git automatically stages every tracked file before the commit

*Remember, commit records the snapshot you set up in the staging area. Each commit basically is a new snapshot of your project*

# Removing a file

If you simply use *rm* command to remove a file, it still shows up under the unstaged files in the git status output. Instead we want:

```
git rm <file name>
```

Removes the file from **both** the staging area and your working directory

```
git rm --cached <file name>
```

Removes the file from **only** the staging area, but keeps it in your working directory

# History Inspection

*There are many powerful options to this command, to learn more check out the documentation.*

```
git log
```

Shows all the commits made

```
git log -p/--patch

git log -p -3
```

Shows all the differences introduced in each commit with its previous one (-3 here limits the output to the last three entries)

```
git log --stat

git log -- path/to/file
```

w the log about that specific file

# Undoing things

Things don’t always go as planned, we might need to undo things from time to time. However, we need to be careful, otherwise we might lose some information !

If we forgot to add some stuff into a commit, or we messed up the commit message, we should do:

```
git commit --amend
```

Takes your staging area and uses it for commit, if you no changes were made, then only the commit message will change. Or if you have forgotten something do the following:

```
git commit -m ‘my commit message’

git add forgotten_file

git commit --amend
```

*You will only have one commit this way, the new one replaces the original; old one* ***won’t*** *be in history*

# Undoing things

What if we want to unstage a file?

```
git reset HEAD file_name
```

What if we want to undo a modification made to a file?

```
git checkout -- file_name
```

**Important: git checkout is dangerous, any local changes to that file will be gone !!!**

From version 2.23.0 on, we can use *git restore* to do the above separately as follows:

```
git restore --staged file_name
```

```
git restore file_name
```

**Important: git restore is also dangerous, any local changes to that file will be gone !!!**

***Don't use these commands, unless you absolutely know that you don't want those local changes!***

# Working with remote

Collaboration is a big part of using Git. We might need to host our code on the Internet or network somewhere. We need to be able to push and pull data from those remote hosts (note: this ‘remote’ can also be your own machine).

To check for the remote servers:

```
git remote
```

```
git remote -v
```

*You should at least see ‘origin’ if you cloned the repo from somewhere else, Git automatically adds this for you.* *-v* *also shows the URLs for each shortname*

# Working with remote

But what if we want to add a new remote ?

```
git remote add myremote https://github.com/somewhere/somerepo
```

Now, we have set up a new remote, in order to get data from that remote, we do:

```
git fetch myremote
```

*This pulls down all the data from that remote, and we now have references to all the branches from that remote. Note this only download the data you are missing, doesn't merge automatically*

# Working with remote

Once we want to share the code in the remote host, we need to push it:

```
git push <remote> <branch>

#example:

git push origin master
```

To see information about a specific remote:

```
git remote show <remote>

#example

git remote show origin
```

Other commands, rename, remove:

```
git remote rename myremote
myremote2

git remote remove myremote
```

# Thanks!